

Rootless Docker Setup with XFS-backed Storage

Objective

Set up **rootless Docker for a user** with:

- Proper UID/GID mapping
 - Dedicated **XFS image-backed storage**
 - Isolated Docker root directory
 - Verified working container runtime
-



Prerequisites

- Ubuntu system (22.04 / 24.04 / similar)
 - Root access
 - Existing user (e.g. `punith.m`)
-



Step 1: Validate User

```
1 id <username>
```

If user does not exist then inform the user to contact the devops team and resolve it.



Step 2: Install Dependencies

```
1 apt update
2 apt install -y \
3     uidmap \
4     slirp4netns \
5     fuse-overlayfs \
6     ca-certificates \
7     curl \
8     gnupg \
9     lsb-release
```



Step 3: Enable Required Kernel Module

```
1 modprobe nf_tables
2 echo "nf_tables" >> /etc/modules ## add if not exists
```



Step 4: Add Docker Repository

```
1 mkdir -p /etc/apt/keyrings
2
3 curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
4 gpg --dearmor -o /etc/apt/keyrings/docker.gpg

1 echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/docker.gpg] \
2 https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" \
3 > /etc/apt/sources.list.d/docker.list

1 apt update
2 apt install -y docker-compose-plugin
```



Step 5: Configure SubUID / SubGID

Check:

```
1 grep ^<username>: /etc/subuid
2 grep ^<username>: /etc/subgid
```

If not present:

```
1 echo "<username>:100000:65536" >> /etc/subuid
2 echo "<username>:100000:65536" >> /etc/subgid
```



Step 6: Enable User Services

```
1 loginctl enable-linger <username>
```



Step 7: Check and Create Required Directories if not exist

```
1 mkdir -p /home/<username>/rootless_docker/bin
2 mkdir -p /home/<username>/.config/systemd/user
3 mkdir -p /home/<username>/.config/docker
```



Step 8: Create XFS Image for Docker Storage

Create image

```
1 truncate -s 50G /bfs/sims/<username>/.xfs_storage.img
```

Format as XFS

```
1 mkfs.xfs /bfs/sims/<username>/.xfs_storage.img
```

Set ownership

```
1 chown <username>:users /bfs/sims/<username>/.xfs_storage.img
2 chmod 600 /bfs/sims/<username>/.xfs_storage.img
```

Step 9: Prepare Docker Storage Path

```
1 mkdir -p /opt/docker_storage/<username>/xfs_storage
2 chown -R <username>:users /opt/docker_storage/<username>
3 chmod -R 700 /opt/docker_storage/<username>
```

Step 10: Mount XFS Image

```
1 mount -o loop /bfs/sims/<username>/.xfs_storage.img \
2 /opt/docker_storage/<username>/xfs_storage
```

Step 11: Create Docker Root Directory

```
1 HOSTNAME=$(hostname)
2
3 mkdir -p /opt/docker_storage/<username>/xfs_storage/$HOSTNAME/docker
4
5 chown -R <username>:users /opt/docker_storage/<username>
```

Step 12: XFS Storage Mount Service (Loop Device on WekaFS)

A **systemd-based service** used to mount an **XFS filesystem backed by a loop device**, where the underlying image file resides on **WekaFS**.

This setup is commonly used for:

- Isolated Docker storage
 - Workload-specific storage environments
 - Controlled filesystem tuning (XFS options)
-

The service performs the following steps:

1. Waits for:
 - WekaFS mount availability
 - Presence of the XFS image file
 - Availability of a loop device
 2. Attaches the image file to a loop device
 3. Mounts the loop device as an XFS filesystem
 4. Continuously monitors the mount and restarts if it is lost
-

Systemd Service File

```

1 [Unit]
2 Description=Mount XFS Storage (Loop on WekaFS)
3 After=network.target
4
5 [Service]
6 Type=simple
7 Restart=always
8 RestartSec=5
9 StandardOutput=journal
10 StandardError=journal
11
12 ExecStart=/bin/bash -x -c '
13 for i in {1..60}; do
14     if mountpoint -q <WEKA_MOUNT> && [ -f <IMAGE_PATH> ] && losetup -f
15     >/dev/null 2>&1; then
16         break;
17     fi;
18     echo "Waiting for Weka + image + loop device..."
19     sleep 2;
20 done;
21 if ! mountpoint -q <TARGET_MOUNT>; then
22     LOOPDEV=$(losetup -f --show <IMAGE_PATH>);
23     mount -o
24     noatime,nodiratime,nodev,nosuid,logbufs=8,logbsize=256k,inode64
25     $LOOPDEV <TARGET_MOUNT>;
26 fi;
27 echo "Mount completed. Starting monitor loop..."
28 while true; do
29     sleep 600;
30     if ! mountpoint -q <TARGET_MOUNT>; then
31         echo "Mount lost! Exiting to trigger restart..."
32         exit 1;
33     fi;
34     echo "Mount healthy";
35 done
36 '
37
38 [Install]
39 WantedBy=multi-user.target

```

1. Create service file

```
1 vi /etc/systemd/system/xfs-storage-<username>.service
```

2. Reload systemd

```
1 systemctl daemon-reexec
2 systemctl daemon-reload
```

3. Enable service

```
1 systemctl enable xfs-storage-<username>.service
```

4. Start service

```
1 systemctl start xfs-storage-<username>.service
```

5. Check status

```
1 systemctl status xfs-storage-<username>.service
2 journalctl -u xfs-storage-<username>.service -f
```



Step 13: Configure Docker Daemon (Rootless)

Create:

```
1 vi /home/<username>/.config/docker/daemon.json
```

Example:

```
1 {
2   "storage-driver": "fuse-overlayfs",
3   "registry-mirrors": ["https://mirror.gcr.io"]
4 }
```



Step 14: Setup Rootless Docker

Switch user:

```
1 su - <username>
```

Set environment:

```
1 export PATH=$HOME/rootless_docker/bin:$PATH
2 export XDG_RUNTIME_DIR=/run/user/$(id -u)
3 export DBUS_SESSION_BUS_ADDRESS=unix:path=$XDG_RUNTIME_DIR/bus
```

Copy and run the binary:

```
1 mkdir -p /home/<username>/rootless_docker/bin
2 cp /bfs/hpc/GENESIS/ansible/rootless_docker/files/docker_bin/*
  /home/<username>/rootless_docker/bin/
3 dockerd-rootless-setuptool.sh install
```



Step 15: Start Docker

```
1 systemctl --user status rootless-docker-<hostname>.service
2 ## if not in running state then start it
3 systemctl --user start rootless-docker-<hostname>.service
```

✓ Step 16: Verify Docker

```
1 | docker info
```

Check:

```
1 | docker info | grep "Docker Root Dir"
```

🔧 Step 17: Test with NGINX

```
1 | docker run -d --name test-nginx -p 7777:80 nginx
```

Test:

```
1 | curl localhost:7777
```

🧹 Step 18: Cleanup

```
1 | docker stop test-nginx
2 | docker rm test-nginx
```

🔍 Step 19: Validate Storage Location

```
1 | docker info | grep "Docker Root Dir"
```

👉 Must point to:

```
1 | /opt/docker_storage/<username>/xfs_storage/<hostname>/docker
```

⚠ Important Notes

♦ XFS behavior

- Can **grow**, cannot shrink
- Use `xfs_growfs` if expanding

♦ 1. Check current size

```
1 | ls -lh /bfs/sims/nishchay.bhutoria/.xfs_storage.img
2 | df -h /opt/docker_storage/nishchay.bhutoria/xfs_storage
```

♦ 2. Extend the image file

Example: increase by 20GB

```
1 truncate -s +20G /bfs/sims/nishchay.bhutoria/.xfs_storage.img
```

👉 OR set absolute size(it should be greater than the current size):

```
1 truncate -s 100G /bfs/sims/nishchay.bhutoria/.xfs_storage.img
```

♦ 3. Refresh loop device (IMPORTANT)

Find loop device:

```
1 losetup -j /bfs/sims/nishchay.bhutoria/.xfs_storage.img
```

Example output:

```
1 /dev/loop5: [...]
```

Then:

```
1 losetup -c /dev/loop5
```

👉 This tells kernel: **file size changed**

♦ 4. Grow XFS filesystem

```
1 xfs_growfs /opt/docker_storage/nishchay.bhutoria/xfs_storage
```

👉 This expands filesystem to full size

♦ 5. Verify

```
1 df -h /opt/docker_storage/nishchay.bhutoria/xfs_storage
```

♦ Rootless limitations

- No privileged containers
 - Port <1024 needs workaround
 - Uses `slirp4netns` (user-space networking)
-

◆ Storage isolation

Each user gets:

```
1 | /opt/docker_storage/<username>/xfs_storage/
```

🚀 Automation (Production Setup)

To perform the above **Rootless Docker setup** in an automated and production-ready manner, use the existing Ansible playbook:

```
1 | /mnt/weka/hpc/GENESIS/ansible/rootless-docker.yaml
```

▶ Run the playbook

```
1 | ansible-playbook -i <inventory_name> -l <limited_clients>  
   | /mnt/weka/hpc/GENESIS/ansible/rootless-docker.yaml -e "username=  
   | <username> xfs_image_size=50G" -f 50
```